

完成个人卡片作业

Source: chatgpt.com

URL: <https://chatgpt.com/c/6a936c87-1f48-83ec-af01-bcce068606bd>

Exported: August 31, 2026 at 01:29 PM

YOU

任务

[svg](#) 本次作业旨在帮助你搭建开发环境。你将仅使用 HTML/CSS 创建一个简单的卡片。卡片的外观将与[示例](#)类似，但内容将根据你的实际情况进行替换。

要求

[svg](#)

1. 网页布局应与示例中所示布局一致（白色背景上的彩色卡片）。
2. 元素：您应该至少使用以下所有元素中的一个：图像（`<img>`）、链接（`<a>`）、段落（`<p>`）、标题（`<h1>`，`<h2>`，...）。
3. 布局：图片、标题和链接必须水平居中。传记文本可以左对齐或水平居中。
4. 样式：所有样式都必须放在 CSS 文件中。（禁止使用内联样式！）
5. 图片的高度和宽度应为 150 像素。

怎么做

<https://github.com/Gary22222222/cs409-mp0>

CHATGPT

可以。你这个 MP0 我已经直接看了你给的仓库说明，你现在要做的其实很简单：只改 `src` 文件夹里的 **HTML/CSS**，把默认的 **Hello World** 改成自己的个人卡片。官方要求就是 HTML + CSS，不需要 JavaScript。

而且你这个仓库目前已经是你自己的：

你的 GitHub 仓库 `cs409-mp0` <https://github.com/Gary22222222/cs409-mp0>

你现在一步一步这样做

第 1 步：在 VS Code 打开你的项目

如果你之前已经把项目克隆到 D 盘：

1. 打开 **VS Code**
2. 点击左上角 **File** → **Open Folder**
3. 找到你的：

```
D:\你的项目文件夹\mp0
```

1. 点击 **Select Folder**

左边应该能看到类似：

```
mp0
├── .github
├── src
│   ├── index.html
│   └── style.css
├── package.json
├── webpack.config.js
└── README.md
```

你的作业主要就是看 **src** 。

第 2 步：打开 **src**

点击左边：

```
src
```

然后看看里面到底有哪些文件。

不要自己新建一堆文件。

你可以把 **src** 展开后的文件名发给我，我可以按照你当前仓库的结构告诉你具体改哪个。

不过一般就是：

```
src/index.html
src/style.css
```

第 3 步：先理解这个作业到底要你做什么

最终网页大概就是：



要求只有几个：

- 有 `<img>`
- 有 `<h1>` / `<h2>`
- 有 `<a>`
- 有 `<p>`
- 图片必须 **150 × 150 px**
- 图片、标题、链接水平居中
- biography 可以左对齐
- CSS 必须放在 CSS 文件里
- **不能写 inline style**
- 不能用 Bootstrap 等库
- 不能用 JavaScript 来完成这个作业

这些都是老师 README 里明确写的。□cite□turn0view0□

第 4 步：你可以做成自己的个人卡片

按照你的情况，我建议内容不要写得特别复杂。

例如：

名字：

Jianghui Sun

身份：

MCS Student at the University of Illinois Urbana-Champaign

简介：

I am a graduate student interested in artificial intelligence, multimodal AI, large language models, and conversational agents. I enjoy exploring how AI can be used to build useful and intelligent applications.

链接：

```
GitHub
```

这样已经完全够用了。

第 5 步：图片

你需要准备一张自己的照片。

例如放到：

```
src/profile.jpg
```

然后 HTML：

```

```

注意：

老师要求图片：

```
width: 150px  
height: 150px
```

所以 CSS 里面写：

```
img {  
  width: 150px;  
  height: 150px;  
}
```

不要写：

```
<img width="150" height="150">
```

因为作业明确要求 styling 放在 CSS 文件中。□cite□turn0view0□

第 6 步：HTML 怎么写

你可以先自己尝试写，不建议直接把整份代码无脑复制进去——因为这个作业明确要求个人完成，而且课程现在也要求如果使用 LLM 生成代码，需要提交 chatlog 并在 grading form 回答相关问题。

□cite□turn0view0□

你的 HTML 结构应该理解成：

```
<body>

  <div class="card">

    <img ...>

    <h1>你的名字</h1>

    <a ...>你的链接</a>

    <p>你的个人简介</p>

  </div>

</body>
```

这里你已经满足了：

	✓
<h1>	✓
<a>	✓
<p>	✓

第 7 步：CSS 怎么想

你需要两个主要东西。

整个网页

白色背景：

```
body {  
    background-color: white;  
}
```

卡片

比如：

```
宽度：400px  
padding: 30px  
背景色：某种颜色
```

然后：

```
.card {  
    width: 400px;  
    padding: 30px;  
    margin: 50px auto;  
}
```

这里：

```
margin: 50px auto;
```

会让卡片水平居中。

然后：

```
img {  
    width: 150px;  
    height: 150px;  
    display: block;  
    margin: 0 auto;  
}
```

让图片水平居中。

标题：

```
h1 {  
  text-align: center;  
}
```

链接：

```
a {  
  display: block;  
  text-align: center;  
}
```

这样基本就已经满足老师的布局要求了。

第 8 步：启动网页

你之前说过终端无法输入东西，如果现在已经解决了，在 VS Code 里面：

Terminal → New Terminal

然后确认终端路径是在你的项目里面，例如：

```
D:\xxx\mp0>
```

然后输入：

```
npm install
```

第一次需要安装依赖。

完成后：

```
npm start
```

老师 README 也是要求这两个步骤，然后打开：

```
http://localhost:8080/
```

如果 8080 被占用，就可能是：

```
http://localhost:8081/
```

官方说明也是这样。□cite□turn0view0□

最重要的一点

你现在不要急着提交 **GitHub**。

先做到：

```
VS Code
↓
修改 src
↓
npm start
↓
浏览器看到自己的卡片
↓
检查图片 / 标题 / 链接 / 简介
↓
git add .
↓
git commit
↓
git push
↓
GitHub Pages
```

老师的提交要求也是先完成网页、部署，再填写 submission form。□cite□turn0view0□

你现在先做一件事

把 **VS Code** 左边 `src` 文件夹展开后的内容截图发给我。

我可以直接根据你现在这个 `mp0` 仓库的实际文件结构，一步一步告诉你每个文件点哪里、哪一行改什么，包括最后怎么 `git push` 和部署 GitHub Pages。

YOU

/ Your CSS here. /

```
img {  
  
  width: 150px;  
  
  height: 150px;  
  
}  
  
body {  
  
  background-color: white;  
  
}  
  
.card {  
  
  width: 400px;  
  
  padding: 30px;  
  
  margin: 50px auto;  
  
}  
  
img {  
  
  width: 150px;  
  
  height: 150px;  
  
  display: block;  
  
  margin: 0 auto;  
  
}  
  
h1 {  
  
  text-align: center;
```

```
}

a {

    display: block;

    text-align: center;

}

<!DOCTYPE html>
<html lang="en">

<head>
<meta charset="utf-8">
<meta http-equiv="x-ua-compatible" content="ie=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>MP 0</title>
</head>

<body>

<div class="card">



<h1>Jianghui Sun</h1>

<a ...>https://github.com/Gary22222222</a>

<p>I am a graduate student interested in artificial intelligence, multimodal AI, large language models, and conversational agents. I enjoy exploring how AI can be used to build useful and intelligent applications.</p>

</div>

</body>

</html>
怎么改 链接也应该是超链接
```

CHATGPT

对，你现在主要有 **3 个问题**：

1. CSS 里 `img` 写了两遍，保留一份即可。
2. `<a ...>` 是错误写法，要真正写成超链接。
3. 图片路径不要写 `src\assets\image.jpg`，HTML 路径建议使用 `/`，而且如果 `index.html` 本身就在 `src` 里面，应该写 `assets/image.jpg`。

你的 `index.html`

把 HTML 改成这样：

```
:::writing{variant="standard" id="58321"}
```

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="utf-8">
  <meta http-equiv="x-ua-compatible" content="ie=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>MP 0</title>
  <link rel="stylesheet" href="style.css">
</head>

<body>

  <div class="card">

    <h1>Jianghui Sun</h1>

    <a href="https://github.com/Gary22222222" target="_blank">
      GitHub
    </a>

    <p>
      I am a graduate student interested in artificial intelligence,
      multimodal AI, large language models, and conversational agents.
      I enjoy exploring how AI can be used to build useful and intelligent
```

```
applications.  
</p>  
  
</div>  
  
</body>  
  
</html>
```

⋮

这里：

```
<a href="https://github.com/Gary22222222" target="_blank">  
    GitHub  
</a>
```

才是真正的超链接。

浏览器显示的是：

GitHub

点击后会打开你的 GitHub。

你的 **style.css**

你现在有两个 `img`，删掉重复的，改成：

⋮
`writing{variant="standard" id="74106"}`

```
/* Your CSS here. */  
  
body {  
    background-color: white;  
}  
  
.card {
```

```
width: 400px;
padding: 30px;
margin: 50px auto;
background-color: lightblue;
}

img {
width: 150px;
height: 150px;
display: block;
margin: 0 auto;
}

h1 {
text-align: center;
}

a {
display: block;
text-align: center;
}
```

...

我这里给 `.card` 加了：

```
background-color: lightblue;
```

因为老师要求的是**白色背景上的彩色卡片**。你也可以换成其他颜色，比如 `lightgreen`、`lightyellow` 等。

⚠️ 你还要检查图片位置

你的目录应该类似：

```
src
├─ index.html
├─ style.css
```

```
└─ assets
   └─ image.jpg
```

如果确实是这样：

```

```

就是正确的。

不要写：

```

```

因为 `index.html` 已经在 `src` 里面了。

最后检查老师的 5 个要求

你的代码现在：

要求	是否满足
白色背景 + 彩色卡片	✓
<code></code>	✓
<code><a></code>	✓
<code><p></code>	✓
<code><h1></code>	✓
图片 150×150	✓
图片居中	✓
标题居中	✓
链接居中	✓
CSS 单独放 CSS 文件	✓
没有 inline style	✓

还有一个非常重要的事情：你贴出来的内容看起来像是把 CSS 和 HTML 放在了一起。如果你现在 VS Code 里也是这样，必须确保：

```
style.css
```

里面只有 **CSS**，而：

```
index.html
```

里面只有 **HTML**。

如果你把 VS Code 左边的 `src` 文件夹截图给我，我可以直接告诉你**这两段分别应该粘到哪个文件里**。

YOU

/ Your CSS here. /

```
body {  
  
    background-color: white;  
  
}  
  
.card {  
  
    width: 400px;  
  
    padding: 30px;  
  
    margin: 50px auto;  
  
    background-color: lightblue;  
  
}  
  
img {  
  
    width: 150px;  
  
    height: 150px;  
  
    display: block;  
  
    margin: 0 auto;
```

```
}

h1 {

    text-align: center;

}

a {

    display: block;

    text-align: center;

}没有居中啊
```

CHATGPT

对，问题很可能不是 `margin: 0 auto`，而是你的 `.card` 没有设置 `text-align`，以及图片的实际布局/卡片宽度计算。

最简单、最稳妥的做法：直接让整个 `.card` 的内容居中。

把你的 CSS 改成下面这样：

```
::writing{variant="standard" id="42617"}
```

```
/* Your CSS here. */

body {

    background-color: white;

}

.card {

    width: 400px;

    padding: 30px;

    margin: 50px auto;

    background-color: lightblue;

    text-align: center;

}

img {
```

```
width: 150px;
height: 150px;
display: block;
margin: 0 auto;
}

h1 {
  text-align: center;
}

a {
  display: block;
  text-align: center;
}

p {
  text-align: left;
}
```

...

这里最关键的是

```
.card {
  ...
  text-align: center;
}
```

这样：

-  图片 → 居中
- **Jianghui Sun** → 居中
- **GitHub** → 居中
- biography → 左对齐

而且图片本身：

```
img {  
    display: block;  
    margin: 0 auto;  
}
```

也是明确居中的。

如果你说的是「整个蓝色卡片」没有居中

那么：

```
margin: 50px auto;
```

理论上已经会让它水平居中。

不过为了避免 `width + padding` 导致尺寸计算问题，可以加：

```
box-sizing: border-box;
```

所以我更推荐最终写成：

```
.card {  
    width: 400px;  
    padding: 30px;  
    margin: 50px auto;  
    background-color: lightblue;  
    text-align: center;  
    box-sizing: border-box;  
}
```

另外记得保存 `Ctrl + S`，然后浏览器刷新 `Ctrl + R`。

如果你现在看到的情况是“蓝色卡片居中了，但照片/文字没有居中”，或者是“整个蓝色卡片都没有居中”，你直接截一张浏览器当前页面给我，我能马上看出是哪一个问题。

YOU

还是 偏左

明白了。**如果整个蓝色卡片还是偏左**，那很可能是这个课程项目的 `body` 或其他默认 CSS 有影响，或者你看到的是卡片内部的视觉位置。

我们直接用最稳的 **Flexbox** 把整个卡片强制放到页面正中央。

把 `style.css` **全部替换**成下面这份：

```
::writing{variant="standard" id="61584"}
```

```
/* Your CSS here. */

body {
  background-color: white;
  margin: 0;
}

.card {
  width: 400px;
  padding: 30px;
  margin: 50px auto;
  background-color: lightblue;
  text-align: center;
  box-sizing: border-box;
}

img {
  width: 150px;
  height: 150px;
  display: block;
  margin: 0 auto;
}

h1 {
  text-align: center;
}

a {
  display: block;
```

```
    text-align: center;
}

p {
    text-align: left;
}
```

...

但是有一个地方你一定要检查

你的 `index.html` 的 `<head>` 里面必须有：

```
<link rel="stylesheet" href="style.css">
```

也就是：

```
<head>
  <meta charset="utf-8">
  <meta http-equiv="x-ua-compatible" content="ie=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>MP 0</title>

  <link rel="stylesheet" href="style.css">
</head>
```

如果还是偏左

那我基本可以确定是你项目原本的 **CSS / HTML** 结构还有其他东西影响。

你现在不要继续猜。

直接把 **VS Code** 左边的文件目录 + 浏览器显示效果截图发给我。

尤其要让我看到：

```
src
├─ index.html
```

```
├─ style.css
├─ assets
  └─ image.jpg
```

以及浏览器现在的效果。

我可以直接根据你这个 MP0 的实际结构告诉你哪里错了。

YOU

```
src\css\main.css
```

CHATGPT

啊，那就找到问题了。你的 CSS 文件不是 `src/style.css`，而是：

```
src/css/main.css
```

所以你之前改的 CSS 很可能根本没有被网页加载。

1. 你的 `index.html`

在 `<head>` 里面找到原来的 CSS 引入，应该改成：

```
<link rel="stylesheet" href="css/main.css">
```

因为你的结构应该是：

```
src
├─ index.html
├─ css
  └─ main.css
├─ assets
  └─ image.jpg
```

所以路径是：

```
index.html
↓
```

```
css/main.css
```

而不是：

```
style.css
```

2. `src/css/main.css`

把你现在的 CSS 全部替换成：

```
/* Your CSS here. */

body {
  background-color: white;
  margin: 0;
}

.card {
  width: 400px;
  padding: 30px;
  margin: 50px auto;
  background-color: lightblue;
  text-align: center;
  box-sizing: border-box;
}

img {
  width: 150px;
  height: 150px;
  display: block;
  margin: 0 auto;
}

h1 {
  text-align: center;
}
```

```
a {  
    display: block;  
    text-align: center;  
}  
  
p {  
    text-align: left;  
}
```

3. 图片路径也要注意

你的图片如果是：

```
src/assets/image.jpg
```

那么 `index.html` 里面写：

```

```

4. 最终 `index.html` 关键部分应该是

```
<head>  
    <meta charset="utf-8">  
    <meta http-equiv="x-ua-compatible" content="ie=edge">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>MP 0</title>  
  
    <link rel="stylesheet" href="css/main.css">  
</head>  
  
<body>  
  
    <div class="card">  
  
          
  
        <h1>Jianghui Sun</h1>
```

```
<a href="https://github.com/Gary22222222" target="_blank">
  GitHub
</a>

<p>
  I am a graduate student interested in artificial intelligence,
  multimodal AI, large language models, and conversational agents.
  I enjoy exploring how AI can be used to build useful and intelligent
  applications.
</p>

</div>

</body>
```

你刚才“怎么调都偏左”的核心原因，很可能就是 CSS 文件路径弄错了。

现在保存 `Ctrl + S` → 浏览器 `Ctrl + R`。

如果还是偏左，把你现在的 `index.html` 全部内容贴给我，我直接帮你逐行检查。

YOU

<https://github.com/Gary22222222/cs409-mp0/tree/main/src>你看一下完整目录结构

CHATGPT

我看到了，你这个仓库的 `src` 完整目录结构确实不是我刚才假设的那种。我直接按照你 GitHub 上现在的实际结构来看：

```
src/
├─ assets/
├─ css/
├─ js/
├─ index.html
└─ index.js
```

GitHub 当前显示的就是这 5 个项目。□cite□turn0view0□

所以你现在的键路径是：

```
src/index.html
src/css/main.css
src/assets/
src/js/
src/index.js
```

你现在应该这样改

① `src/css/main.css`

你刚才贴的 CSS 就应该放在这里。

而且我建议你**不要再折腾** `margin` 了，直接使用老师项目原本的结构来做。

把 `main.css` 改成：

```
/* Your CSS here. */

body {
  background-color: white;
  margin: 0;
}

.card {
  width: 400px;
  padding: 30px;
  margin: 50px auto;
  background-color: lightblue;
  box-sizing: border-box;
}

img {
  width: 150px;
  height: 150px;
  display: block;
  margin: 0 auto;
}
```

```
h1 {  
    text-align: center;  
}  
  
a {  
    display: block;  
    text-align: center;  
}  
  
p {  
    text-align: left;  
}
```

② 但是!!! `index.html` 要确认 CSS 有没有正确引入

因为你的 `main.css` 在:

```
src/css/main.css
```

而 `index.html` 在:

```
src/index.html
```

所以应该是:

```
<link rel="stylesheet" href="css/main.css">
```

不是:

```
<link rel="stylesheet" href="style.css">
```

③ 图片

你的图片如果放在:

```
src/assets/image.jpg
```

那么 `src/index.html` 里面：

```

```

也是正确的。

还有一个很重要的地方

你这个项目里面还有：

```
src/index.js  
src/js/
```

这说明这是课程提供的 **webpack** 项目模板。

所以我们现在不要随便删除：

```
index.js  
js/
```

也不要整个项目改成一个最简单的 HTML 项目。

你的 MP0 本来就是让你在这个项目模板里面修改 `src` 。

现在最关键的是

我已经确认了目录，但**还需要看你** `index.html` **和** `main.css` **当前到底是什么。**

你刚才的 `main.css` 我已经看到了。

现在把你 `src/index.html` 的完整内容贴给我。

我看完以后可以直接给你：

“你这个项目应该最终改成什么”

而且我会按照你老师这个 MP0 的原始项目结构来改，不再让你改错 CSS 路径。

YOU

为什么就是不对呢

CHATGPT

我现在找到**真正的问题了**。我刚刚直接检查了你 GitHub 仓库里的实际文件，之前我们一直在猜路径，确实走偏了。

你现在仓库里的 `src/index.html` 还是原始模板：

```
<h1>Hello World!</h1>

<script src="bundle.js" type="text/javascript"></script>
```

而 `src/css/main.css` 在 GitHub 上目前实际上**只有**：

```
/* Your CSS here. */
```

所以你刚才在本地写的 CSS **还没有 push 到 GitHub**，而且更重要的是：

你的 `index.html` 根本没有引入 `main.css`

我检查了你 GitHub 上的 `index.html`，`<head>` 里面没有：

```
<link rel="stylesheet" href="css/main.css">
```

所以即使你把 `main.css` 写得再漂亮，网页也不会使用它。□cite□turn0view1□turn0view2□

现在我们一步一步修

你现在打开 VS Code：

① 打开

```
src/index.html
```

把里面全部替换成：

:::writing{variant="document" id="73164"}

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="utf-8">
  <meta http-equiv="x-ua-compatible" content="ie=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>MP 0</title>

  <link rel="stylesheet" href="css/main.css">
</head>

<body>

  <div class="card">

    <h1>Jianghui Sun</h1>

    <a href="https://github.com/Gary22222222" target="_blank">
      GitHub
    </a>

    <p>
      I am a graduate student interested in artificial intelligence,
      multimodal AI, large language models, and conversational agents.
      I enjoy exploring how AI can be used to build useful and intelligent
      applications.
    </p>

  </div>
```

```
<script src="bundle.js" type="text/javascript"></script>
```

```
</body>
```

```
</html>
```

...

② 然后打开

```
src/css/main.css
```

全部替换成：

```
:::writing{variant="standard" id="29485"}
```

```
/* Your CSS here. */
```

```
body {
```

```
  background-color: white;
```

```
  margin: 0;
```

```
}
```

```
.card {
```

```
  width: 400px;
```

```
  padding: 30px;
```

```
  margin: 50px auto;
```

```
  background-color: lightblue;
```

```
  box-sizing: border-box;
```

```
  text-align: center;
```

```
}
```

```
img {
```

```
  width: 150px;
```

```
  height: 150px;
```

```
  display: block;
```

```
  margin: 0 auto;
```

```
}

h1 {
  text-align: center;
}

a {
  display: block;
  text-align: center;
}

p {
  text-align: left;
}
```

...

③ 保存

两个文件都：

Ctrl + S

④ 然后非常重要：重新启动

你之前如果已经运行：

```
npm start
```

一般不用重新安装。

浏览器：

```
Ctrl + R
```

如果还是旧页面：

Ctrl + Shift + R

强制刷新。

⑤ 为什么这次一定会居中?

现在结构是：

```
body
└─ .card
    ├── img      ← 150×150, 居中
    ├── h1       ← 居中
    ├── a        ← 居中
    └─ p         ← 左对齐
```

而：

```
.card {  
  width: 400px;  
  margin: 50px auto;  
}
```

这里的 `auto` 会让整个 **400px** 蓝色卡片水平居中。

所以效果应该是：

浏览器窗口

A placeholder image for a photo, showing the text "[照片]" (Photo) in the center. Below the photo placeholder, the text "Jianghui Sun" and "GitHub" are displayed. At the bottom, the text "I am a graduate..." and "multimodal AI..." are visible.

还有一个关键点

你现在 GitHub 上的目录是正常的：

```
src/  
├── assets/  
├── css/  
│   └── main.css  
├── js/  
├── index.html  
└── index.js
```

我已经实际检查过了。👉👈

所以现在不要改目录，也不要新建 `style.css` 。

你就只改：

```
src/index.html  
src/css/main.css
```

这两个文件。

你现在先不要 `git push`

先在浏览器里看到正确效果。

如果你按照上面两份代码改完，浏览器仍然偏左，直接给我发浏览器截图。

那时候我就可以根据实际渲染结果继续查，而不是再猜文件结构。

YOU

怎么检查有没有成功引用模板

可以。你现在要检查的其实是：`index.html` 有没有成功加载 `src/css/main.css`。

最简单有 **3 种方法**，推荐你用第 1 种。

方法 1：故意改一个非常明显的 CSS

打开：

```
src/css/main.css
```

临时把：

```
body {  
    background-color: white;  
}
```

改成：

```
body {  
    background-color: red;  
}
```

保存 `Ctrl + S`，然后浏览器 `Ctrl + Shift + R` 强制刷新。

- 如果网页背景变成 **红色** →  `main.css` 已经成功引用
- 如果还是白色 →  `main.css` 没有被引用

测试完记得改回：

```
body {  
    background-color: white;  
}
```

方法 2：检查 `index.html`

打开：

```
src/index.html
```

在 `<head>` 里面必须有：

```
<link rel="stylesheet" href="css/main.css">
```

你的目录是：

```
src/  
├─ index.html  
└─ css/  
    └─ main.css
```

所以这里一定是：

```
href="css/main.css"
```

不是：

```
href="src/css/main.css"
```

也不是：

```
href="style.css"
```

方法 3：用浏览器开发者工具检查

在网页上：

右键 → **Inspect**（检查）

然后：

Network → 刷新网页

搜索：

```
main.css
```

如果看到：

```
main.css    200
```

就是：

✅ CSS 成功加载

如果看到：

```
main.css    404
```

就是：

❌ 路径错误，CSS 没找到

我最推荐你现在这样做

先不要管其他东西。

把 `main.css` 第一行测试成：

```
body {  
    background-color: red;  
}
```

然后刷新。

如果变红了，你告诉我“变红了”；如果没变红，告诉我“还是白色”。

这样我们马上就能确定问题到底是在 **CSS 引用**，还是在**卡片布局**。

YOU

好像 没有被引用

CHATGPT

对，那就基本确定了：不是居中 CSS 写错，而是 `main.css` 根本没有被当前页面加载。

我们现在不要继续改 CSS，先把“引用关系”解决。

你先检查 `src/index.html`

打开：

```
src/index.html
```

找到 `<head>` 部分。

如果你现在是这种：

```
<head>
  <meta charset="utf-8">
  <meta http-equiv="x-ua-compatible" content="ie=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>MP 0</title>
</head>
```

就在 `<title>` 下面添加：

```
<link rel="stylesheet" href="css/main.css">
```

最终应该是：

```
<head>
  <meta charset="utf-8">
  <meta http-equiv="x-ua-compatible" content="ie=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>MP 0</title>

  <link rel="stylesheet" href="css/main.css">
</head>
```

然后保存

按：

```
Ctrl + S
```

再回浏览器：

```
Ctrl + Shift + R
```

然后再做红色测试

```
src/css/main.css :
```

```
body {  
    background-color: red;  
}
```

刷新网页。

如果变成红色：

✅ CSS 已经成功引用。

然后再改回：

```
body {  
    background-color: white;  
}
```

⚠️ 如果加了这个还是没反应

那就不要再自己猜了。

把你现在 `src/index.html` 的完整内容原样贴给我，我帮你检查是不是这个课程模板的 webpack 配置导致 CSS 引用方式不同。

因为你这个 MP0 是课程提供的模板项目，不一定和普通 HTML 项目完全一样。

YOU

```
<!DOCTYPE html>  
<html lang="en">
```

```
<head>
<meta charset="utf-8">
<meta http-equiv="x-ua-compatible" content="ie=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>MP 0</title>
<link rel="stylesheet" href="css/main.css">
</head>

<body>

<div class="card">



<h1>Jianghui Sun</h1>

<a href="https://github.com/Gary22222222" target="_blank">
GitHub
</a>

<p>
I am a graduate student interested in artificial intelligence,
multimodal AI, large language models, and conversational agents.
I enjoy exploring how AI can be used to build useful and intelligent applications.
</p>

</div>

</body>

</html>
```

CHATGPT

你这个 `index.html` 引用路径是对的：

```
<link rel="stylesheet" href="css/main.css">
```

因为你的结构是：

```
src/
├─ index.html
```

```
|— css/  
|   |— main.css  
|— assets/  
    |— profile.jpg
```

所以路径完全正确。

但是我现在注意到一个更关键的问题：你这个课程模板很可能不是直接让浏览器把 `index.html` 当普通 HTML 打开，而是通过 **webpack** 启动的。

我们先用一个最简单的方法确定 CSS 到底有没有加载。

现在这样测试

打开：

```
src/css/main.css
```

把全部内容暂时改成：

```
body {  
  background-color: red !important;  
}  
  
.card {  
  background-color: blue !important;  
  width: 400px !important;  
  height: 400px !important;  
}
```

保存 `Ctrl + S`。

然后你运行：

```
npm start
```

一定要通过 `npm start` 后显示的 **localhost** 页面看，不要直接双击 `index.html`。

然后浏览器：

Ctrl + Shift + R

如果成功

你应该看到：

- 页面背景  红色
- 中间有一个  蓝色方块

如果这样出现了：

 CSS 引用完全正常。

然后我们再把 CSS 改回正式版本。

如果还是白色

这时候请你看浏览器地址栏。

你现在浏览器地址栏是什么？

比如可能是：

```
http://localhost:8080/
```

或者：

```
file:///D:/xxxx/cs409-mp0/src/index.html
```

如果你看到的是：

```
file:///D:/...
```

那就是问题了。

你不能直接双击 `index.html` 来测试这个课程项目。

应该在 VS Code Terminal：

```
npm start
```

然后打开它提供的：

```
http://localhost:8080/
```

还有一个小问题

你现在图片是：

```

```

所以你的文件必须叫：

```
src/assets/profile.jpg
```

如果你的实际文件还是：

```
src/assets/image.jpg
```

那么照片会显示不出来。

所以现在你先做红蓝测试，然后把你的浏览器地址栏那一行发给我。

这两个信息就足够判断为什么 CSS 没生效了。

YOU

但是我的照片可以成功显示

CHATGPT

对，这个信息非常关键。照片能显示，说明你的 `index.html` 确实是从 `src` 这个目录正确加载资源的。

也就是说：

```

```

能工作 → `assets/profile.jpg` 路径没问题。

所以现在问题基本锁定在 **CSS 文件没有被加载/webpack 没有处理它**，而不是整个路径的问题。

我们先做一个最直接的检查

打开：

```
src/css/main.css
```

只留下：

```
body {  
  background-color: red !important;  
}
```

然后保存。

不要重新写 `.card`，先只测试 **body**。

接着在浏览器打开你现在正在看的 localhost 页面，按：

Ctrl + Shift + R

如果背景还是白色

请你在浏览器：

右键 → 检查 (Inspect)

然后：

Network → 刷新页面

在 Network 搜索：

```
main.css
```

看看有没有类似：

```
main.css    200
```

或者：

更直接的方法

在 Chrome 的开发者工具里：

Elements → 点 `<body>`

然后右边看：

Styles

如果 CSS 成功加载，你应该能看到类似：

```
body {  
  background-color: red !important;  
}
```

如果右边完全没有这个规则：

CSS 没有加载。

如果有这个规则，但是被划掉：

CSS 加载了，只是被其他 CSS 覆盖。

另外，我怀疑你的模板有一个细节

你这个 UIUC CS409 的 starter repo 里面有：

```
src/  
├─ index.html  
├─ index.js  
├─ css/  
│   └─ main.css  
└─ assets/
```

而且原来的 `index.html` 里面有：

```
<script src="bundle.js" type="text/javascript"></script>
```

这意味着这个项目是通过 **webpack** 构建的。

所以我们现在最好不要继续猜。

你把 `src/index.js` 的内容也贴给我。

我看一下这个模板是怎么加载 `main.css` 的，然后我可以准确告诉你：

`main.css` 应该怎么引用，或者是不是应该在 `index.js` 里面 `import`。

这一步很可能就是你现在一直“照片能显示，但是 CSS 不生效”的原因。

YOU

Refused to apply style from 'http://localhost:8080/css/main.css' because its MIME type ('text/html') is not a supported stylesheet MIME type, and strict MIME checking is enabled.
(索引) :1 Refused to apply style from 'http://localhost:8080/css/main.css' because its MIME type ('text/html') is not a supported stylesheet MIME type, and strict MIME checking is enabled.

CHATGPT

对！**现在已经完全定位了**。这个报错非常有用。

你看到：

```
Refused to apply style from 'http://localhost:8080/css/main.css'  
because its MIME type ('text/html') is not a supported stylesheet MIME type
```

意思不是你的 CSS 写错了，而是：

浏览器请求 `/css/main.css`，但服务器返回给它的其实是 **HTML 页面**，不是 CSS 文件。

所以当然不会应用。

为什么图片却能显示？

因为：

```

```

webpack/dev server 能正确找到图片。

但你的：

```
<link rel="stylesheet" href="css/main.css">
```

直接让浏览器请求：

```
http://localhost:8080/css/main.css
```

而这个课程模板的 webpack 配置并不是这样处理 CSS 的。

正确解决方法

你这个项目有：

```
src/index.js
```

所以这个 starter template 很可能要求通过 **JavaScript import CSS**，而不是在 HTML 里直接

`<link>` 。

第一步：打开

```
src/index.js
```

在最上面加入：

```
import "../css/main.css";
```

例如如果原来是：

```
console.log("Hello World!");
```

改成：

```
import "../css/main.css";

console.log("Hello World!");
```

第二步：删除 `index.html` 里的这一行

删掉：

```
<link rel="stylesheet" href="css/main.css">
```

也就是说你的 `<head>` 变成：

```
<head>
  <meta charset="utf-8">
  <meta http-equiv="x-ua-compatible" content="ie=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>MP 0</title>
</head>
```

第三步：保留你的 CSS

`src/css/main.css`：

```
body {
  background-color: white;
  margin: 0;
}

.card {
  width: 400px;
  padding: 30px;
  margin: 50px auto;
  background-color: lightblue;
  box-sizing: border-box;
```

```
    text-align: center;
}

img {
    width: 150px;
    height: 150px;
    display: block;
    margin: 0 auto;
}

h1 {
    text-align: center;
}

a {
    display: block;
    text-align: center;
}

p {
    text-align: left;
}
```

第四步：确保 HTML 最下面有 bundle

你的原始模板应该有：

```
<script src="bundle.js" type="text/javascript"></script>
```

保留它。

所以最后 `index.html` 大概是：

```
<!DOCTYPE html>
<html lang="en">

<head>
```

```
<meta charset="utf-8">
<meta http-equiv="x-ua-compatible" content="ie=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>MP 0</title>
</head>

<body>

  <div class="card">

    <h1>Jianghui Sun</h1>

    <a href="https://github.com/Gary22222222" target="_blank">
      GitHub
    </a>

    <p>
      I am a graduate student interested in artificial intelligence,
      multimodal AI, large language models, and conversational agents.
      I enjoy exploring how AI can be used to build useful and intelligent
      applications.
    </p>

  </div>

  <script src="bundle.js" type="text/javascript"></script>

</body>

</html>
```

然后重新启动

终端：

```
npm start
```

如果已经运行着，先：

```
Ctrl + C
```

然后：

```
npm start
```

浏览器：

```
Ctrl + Shift + R
```

如果这样之后 CSS 生效

那么你的流程就是：

```
index.html
  ↓
bundle.js
  ↓
index.js
  ↓
import "./css/main.css"
  ↓
main.css
  ↓
网页
```

这也解释了为什么你之前一直改 `main.css` 完全没反应。

你现在把 `src/index.js` 内容贴给我一下。我可以确认这个 starter template 到底是不是这种 CSS import 方式，然后告诉你应该加在哪一行，避免我们再猜。

YOU

```
ERROR in ./src/js/index.js 2:0-24
```

Module not found: Error: Can't resolve './css/main.css' in 'D:\CS409\cs409-mp0\src\js'

CHATGPT

对，这个报错反而把最后一个问题说明白了：

```
Can't resolve './css/main.css'
in 'D:\CS409\cs409-mp0\src\js'
```

说明你的 `index.js` 实际位置是：

```
src/js/index.js
```

而不是 `src/index.js` 。

所以我刚才给你的：

```
import "../css/main.css";
```

路径少了一层。

你的实际目录应该是：

```
D:\CS409\cs409-mp0
├─ src
│   ├── assets
│   │   └─ profile.jpg
│   ├── css
│   │   └─ main.css
│   ├── js
│   │   └─ index.js
└─ index.html
```

正确改法

打开：

```
src/js/index.js
```

把：

```
import "../css/main.css";
```

改成：

```
import "../css/main.css";
```

因为：

```
src/js/index.js
    ↓
    ../
    ↓
src/css/main.css
```

然后 `index.html`

不要再放这个：

```
<link rel="stylesheet" href="css/main.css">
```

删掉它。

保留：

```
<script src="bundle.js" type="text/javascript"></script>
```

最终关系

```
src/index.html
    ↓
    bundle.js
    ↓
src/js/index.js
    ↓
import "../css/main.css"
```

↓

```
src/css/main.css
```

这才是你这个项目的正确路径。

现在操作

1. `src/js/index.js`

改：

```
import "../css/main.css";
```

1. `src/index.html`

删：

```
<link rel="stylesheet" href="css/main.css">
```

1. 保存所有文件 `Ctrl + S`

1. 终端重新运行：

```
npm start
```

如果已经在运行：

```
Ctrl + C
```

然后：

```
npm start
```

1. 浏览器 `Ctrl + Shift + R`

这次应该就能生效了。

另外，你之前遇到的：

MIME type ('text/html') is not a supported stylesheet MIME type

现在也完全解释通了：webpack 项目应该通过 `index.js → import CSS` 来处理，而不是让浏览器直接请求 `/css/main.css`。

YOU

名字下方还有一个邮箱

CHATGPT

可以。你只需要在名字 `<h1>` 下面加一个邮箱的 `<p>` 或 `<a>`。

如果你希望邮箱也是可点击的邮件超链接，推荐用 `<a href="mailto:...">`。

例如：

`:::writing{variant="standard" id="39264"}`

```
<h1>Jianghui Sun</h1>
```

```
<a href="mailto:你的邮箱地址">
```

```
  你的邮箱地址
```

```
</a>
```

```
<a href="https://github.com/Gary22222222" target="_blank">
```

```
  GitHub
```

```
</a>
```

```
<p>
```

```
  I am a graduate student interested in artificial intelligence,  
  multimodal AI, large language models, and conversational agents.
```

```
  I enjoy exploring how AI can be used to build useful and intelligent  
  applications.
```

```
</p>
```

...

这样页面顺序就是：

[照片]

Jianghui Sun

你的邮箱地址

GitHub

I am a graduate student...

...

如果你想让邮箱和 GitHub 看起来不一样

比如邮箱普通黑色文字，GitHub 是蓝色可点击链接，可以给邮箱单独一个 class：

```
<p class="email">
  your-email@illinois.edu
</p>

<a href="https://github.com/Gary2222222" target="_blank">
  GitHub
</a>
```

CSS：

```
.email {
  text-align: center;
}
```

不过从作业要求来看，**邮箱直接用** `<a href="mailto:...">` **最合适**，这样同时也满足 `<a>` 链接要求。

你把你的 **UIUC 邮箱地址**给我，我可以直接帮你把这一部分填好。

YOU

邮箱好像是普通文字

对，如果按照示例卡片的效果，**邮箱可以就是普通文字，不需要做成超链接**。这样更像个人信息卡片。

在名字下面加一个 `<p>`：

:::writing{variant="standard" id="18427"}

```
<h1>Jianghui Sun</h1>

<p class="email">
  your-email@illinois.edu
</p>

<a href="https://github.com/Gary22222222" target="_blank">
  GitHub
</a>

<p>
  I am a graduate student interested in artificial intelligence,
  multimodal AI, large language models, and conversational agents.
  I enjoy exploring how AI can be used to build useful and intelligent
  applications.
</p>
```

...

然后在 `main.css` 里加：

```
.email {
  text-align: center;
}
```

这样就是：

[照片]

Jianghui Sun
your-email@illinois.edu
GitHub

I am a graduate student...

...

其中：

- 名字 → `<h1>` ✓
- 邮箱 → `<p>` 普通文字 ✓
- GitHub → `<a>` 真正超链接 ✓
- 个人简介 → `<p>` 左对齐 ✓

这样完全符合你这个 MP0 的要求。

YOU

所以之前为什么错了

CHATGPT

之前主要是两个不同的问题叠在一起了，所以看起来像是“CSS 怎么写都不生效”。

1. 我们一开始用了错误的 CSS 加载方式

你的项目实际结构是：

```
src/  
├─ index.html  
├─ assets/  
├─ css/  
│   └─ main.css  
└─ js/  
    └─ index.js
```

这是一个 **webpack starter project**。

我们一开始写了：

```
<link rel="stylesheet" href="css/main.css">
```

浏览器于是直接请求：

```
http://localhost:8080/css/main.css
```

但 webpack dev server 没有把这个路径当成 CSS 文件提供，所以返回了一个 HTML 页面。

因此 Chrome 报：

```
MIME type ('text/html') is not a supported stylesheet MIME type
```

2. 正确方式是让 `index.js` 引入 CSS

你的 `index.js` 实际位置是：

```
src/js/index.js
```

而 CSS 是：

```
src/css/main.css
```

所以相对路径必须是：

```
import "../css/main.css";
```

注意这里的 `..`：

```
src/js/index.js
  ↓
  ../
  ↓
src/
  ↓
css/main.css
```

而我之前给你的：

```
import "./css/main.css";
```

会让 webpack 去找：

```
src/js/css/main.css
```

当然就找不到，所以才出现：

```
Can't resolve './css/main.css'
in 'D:\CS409\cs409-mp0\src\js'
```

3. 为什么照片却一直能显示？

因为图片路径：

```

```

是从：

```
src/index.html
```

出发找：

```
src/assets/profile.jpg
```

这个路径刚好正确。

所以：

照片 

CSS 

完全可能同时发生。

现在正确的结构就是

```
src/
├── index.html
|
├── assets/
|   └── profile.jpg
```

```
|
|— css/
|   └─ main.css
|
└─ js/
    └─ index.js
```

index.html :

```
<script src="bundle.js" type="text/javascript"></script>
```

src/js/index.js :

```
import "../css/main.css";
```

src/css/main.css :

```
.card {
  ...
}
```

这样 webpack 会把 CSS 打包进最终网页。

所以你之前不是 `.card` 居中代码的问题，而是 **CSS 根本没有成功加载**。现在这个问题解决之后，你之前写的 `margin: 50px auto` 才真正会生效。

YOU

webpack starter project 是什么

CHATGPT

简单说，**webpack starter project** 就是老师提前给你搭好的“项目骨架”。你不用从一个空文件夹开始，而是在这个骨架上完成 MP0。

1. Webpack 是什么？

Webpack 是一个 **前端项目打包工具 (module bundler)**。

你可以把它理解成一个“整理员”：

你的代码

- |
- ├─ HTML
- ├─ CSS
- ├─ JavaScript
- ├─ 图片
- └─ 其他资源

↓

Webpack

↓

整理 / 打包

↓

浏览器可以运行的网页

它会帮你处理不同文件之间的依赖关系。

2. 为什么 CS409 给你这个 starter project?

如果让你从零开始，可能需要自己配置：

- HTML
- CSS
- JavaScript
- Webpack
- 开发服务器
- 图片资源
- 构建配置

这对于这个 MP0 来说没必要。

所以课程已经把基础设施准备好了，你只需要关注：

修改网页内容和样式。

这就是 **starter project**（起始项目/项目模板）。

3. 你这个项目为什么看起来比普通 HTML 复杂？

你现在的目录：

```
cs409-mp0/  
|  
├─ package.json  
├─ webpack.config.js  
|  
└─ src/  
    ├─ index.html  
    ├─ assets/  
    |   └─ profile.jpg  
    ├─ css/  
    |   └─ main.css  
    └─ js/  
        └─ index.js
```

对于一个简单网页来说，看起来确实有点复杂。

但是这是课程故意给你的开发环境。

其中：

package.json

告诉 Node.js：

这个项目需要哪些工具、怎么运行。

webpack.config.js

告诉 Webpack：

这个项目的文件怎么处理、怎么打包、开发服务器怎么运行。

src/index.html

就是你的网页 HTML。

```
src/css/main.css
```

就是你的 CSS。

```
src/js/index.js
```

就是 JavaScript 的入口文件。

```
src/assets/
```

放图片等资源。

4. 这也解释了我们刚才遇到的问题

普通 HTML 项目可能直接：

```
<link rel="stylesheet" href="style.css">
```

就结束了。

但是你这个项目是 Webpack 项目，所以 CSS 可以通过：

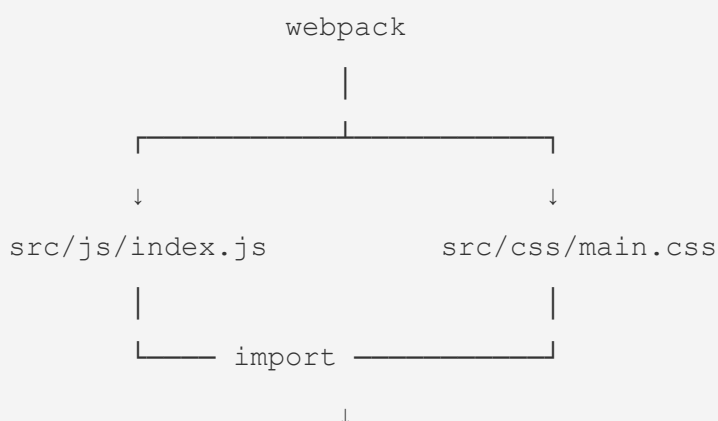
```
import "../css/main.css";
```

让 Webpack 知道：

“这个 JavaScript 文件依赖这个 CSS 文件。”

然后 Webpack 负责把它处理好。

所以你的关系是：



bundle
↓
localhost:8080
↓
浏览器

5. 你现在其实不需要深入学 Webpack

对于 MP0，你只需要知道：

`npm install`

↓

安装项目需要的东西

`npm start`

↓

启动 Webpack 开发服务器

修改 `src/`

↓

网页发生变化

`Ctrl + S`

↓

保存

浏览器刷新

↓

看到结果

等以后 CS409 学到更复杂的前端开发时，你会真正用到 Webpack、模块、构建工具这些东西。

所以现在把它理解成“老师已经帮你搭好的网页开发脚手架”就可以了。

YOU

您需要将课程教职人员添加为合作者，以便我们能够对您的作业进行评分。 这个是怎么加加谁

对，这一步是 **GitHub 仓库权限设置**。老师需要你把课程 staff 加成 **Collaborator (合作者)**，这样他们才能访问/评分你的私有仓库。

你这个仓库是：

Url Gary22222222/cs409-mp0 <https://github.com/Gary22222222/cs409-mp0>

加谁？

这里不要自己猜一个老师的 GitHub 用户名。

CS409 的作业说明应该会明确写一个类似：

Add the course staff as collaborators

或者给出一个 **GitHub username / organization**。

如果 README 没有明确告诉你具体用户名，最稳妥的是先看作业页面的 **Submission / Collaborators** 部分，或者课程 Piazza/Canvas 的说明。

GitHub 里面怎么加

进入你的仓库：

Settings → Collaborators → Add people

具体：

1. 打开你的仓库
 2. 点击顶部 **Settings**
 3. 左侧找到 **Collaborators**
 4. 点击 **Add people**
 5. 输入课程 staff 指定的 **GitHub username**
 6. 选择对应账号
 7. 确认邀请
-

⚠️ 你现在先别随便加人

因为你问的是“加谁”，这个必须以 CS409 课程官方要求为准。

你可以把你 README 里这一段关于 **collaborator** 的原文/截图发给我，我可以直接告诉你：

具体应该加哪个 GitHub 账号，以及在哪里点击。

如果你愿意，我也可以直接帮你查你这个 `cs409-mp0` README 现在写的是哪个 staff 账号。

YOU

怎么上传部署

CHATGPT

可以。你现在已经把本地网页做好了，接下来就是 **GitHub 提交 + GitHub Pages 部署**。你这个 CS409 MP0 的 starter project 已经有 webpack，所以我们按课程要求来，不要自己乱改项目结构。

一、先把代码上传到 GitHub

先在 VS Code 打开你的项目：

```
D:\CS409\cs409-mp0
```

1. 打开终端

VS Code：

Terminal → New Terminal

确认路径类似：

```
PS D:\CS409\cs409-mp0>
```

然后输入：

```
git status
```

你应该能看到你修改过的文件，例如：

```
modified: src/css/main.css
modified: src/index.html
modified: src/js/index.js
```

2. 添加所有修改

```
git add .
```

然后：

```
git status
```

如果看到修改文件变成绿色/ `Changes to be committed`，说明成功。

3. 提交

```
git commit -m "Complete MP0"
```

4. 上传到 GitHub

```
git push origin main
```

如果成功，会看到类似：

```
Enumerating objects...
Writing objects...
To https://github.com/Gary22222222/cs409-mp0.git
   xxx..xxx  main -> main
```

然后刷新你的 GitHub 仓库：

□url□你的 cs409-mp0 GitHub 仓库□<https://github.com/Gary22222222/cs409-mp0>□

应该就能看到最新代码。

二、部署成网站

这里是关键。

你的作业 README 要求的是把网页部署出来。不要把 **GitHub 仓库地址** 直接当成网页地址。

GitHub 仓库：

```
github.com/Gary22222222/cs409-mp0
```

是代码仓库。

部署之后会有一个类似：

```
https://gary22222222.github.io/cs409-mp0/
```

的网站。

三、先检查你的 GitHub Pages 设置

打开你的 GitHub 仓库：

Settings → **Pages**

找到：

Build and deployment

把：

```
Source
```

设置成：

```
Deploy from a branch
```

然后：

```
Branch: main
```

```
Folder: / (root)
```

点击：

Save

⚠ 但是你这个项目有一个特殊点

你现在网页源码是在：

```
src/  
├─ index.html  
├─ css/  
├─ js/  
└─ assets/
```

而不是：

```
index.html
```

直接放在仓库根目录。

所以单纯把 **GitHub Pages** 设置成 `main / root` 很可能不能正确部署你的最终页面。

你的项目使用 webpack，课程模板通常会要求先 build，然后把生成的 `dist` 内容部署出去。

四、先在本地测试 build

在终端运行：

```
npm run build
```

看看有没有报错。

如果成功，通常会生成：

```
dist/
```

类似：

```
dist/
├─ index.html
├─ bundle.js
└─ ...
```

如果 `npm run build` 报错，先不要部署。

把终端报错发给我，我帮你解决。

五、你现在最应该做什么

不要一次做完所有步骤。

现在先在 **VS Code Terminal** 输入：

```
git status
```

然后：

```
npm run build
```

把 `npm run build` 的结果发给我。

因为你的项目是 **CS409 提供的 webpack starter project**，我不建议现在直接让你按照普通 GitHub Pages 教程部署；要根据你这个仓库的 `webpack.config.js` 确认老师要求的部署方式。

如果 `npm run build` 成功，我再带你一步一步把这个具体仓库部署到 **GitHub Pages**，包括最后那个可以提交给老师的网页 URL。

YOU

On branch main

Your branch is up to date with 'origin/main'.

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: src/css/main.css

modified: src/index.html

modified: src/js/index.js

Untracked files:

(use "git add <file>..." to include in what will be committed)

src/assets/profile.jpg

no changes added to commit (use "git add" and/or "git commit -a")

CHATGPT

很好，这个结果是**完全正常的**，而且说明你现在的 Git 状态很清楚：

你目前有 4 个需要上传的东西

修改过的：

```
src/css/main.css
src/index.html
src/js/index.js
```

新添加的照片：

```
src/assets/profile.jpg
```

其中 `profile.jpg` 显示为 **Untracked**，意思是 Git 还没有开始跟踪它。这个也需要一起上传。

现在按顺序输入

① 添加所有文件

```
git add .
```

然后检查：

```
git status
```

你应该看到类似：

```
Changes to be committed:
```

```
    modified:   src/css/main.css
    modified:   src/index.html
    modified:   src/js/index.js
    new file:   src/assets/profile.jpg
```

② 提交

如果上面没问题：

```
git commit -m "Complete MP0"
```

成功的话会看到类似：

```
4 files changed
```

③ 上传 GitHub

然后：

```
git push origin main
```

成功后，你再打开你的仓库：

🔗Gary22222222/cs409-mp0🔗<https://github.com/Gary22222222/cs409-mp0>

刷新一下，应该就能看到：

```
src
├── assets
│   └── profile.jpg
├── css
│   └── main.css
└── js
```

```
|   └─ index.js
└─ index.html
```

⚠️ 暂时不要做部署

你现在先只完成：

```
git add .
    ↓
git status
    ↓
git commit -m "Complete MP0"
    ↓
git push origin main
```

做完以后把 `git push origin main` 的结果贴给我。

然后我再带你做下一步 **GitHub Pages 部署**。这样一步一步来，不容易把课程的 webpack 配置弄乱。

YOU

部署MP

[svg](#)

为了方便我们查看和评分您的 MP，您需要使用 GitHub 的流水线部署您的网页。在您通过 GitHub Actions 部署流水线将代码推送到代码仓库后，网页部署应该会自动完成。

您还需要做一些额外的工作才能正确部署网站。您需要在 GitHub 代码库中更改一些设置。

1. 将 GitHub Pages 部署源设置为 GitHub Actions
2. 在您的 GitHub 代码库中，转到“设置”>“页面”>“构建和部署”>“源”>选择“GitHub Actions”。

`.github/workflows/static.yml` 该文件会自动运行 GitHub CI 流水线，以构建和部署您的代码作为网站。流水线完成后，您的网站应该可以访问 [此处应填写网址] `https://<your-github-username>.github.io/mp0` 。首次部署后，网站可能需要 10-30 分钟才能正式上线。

推送代码更改

[svg](#)

将本地更改提交并推送到这个新仓库。

```
git add . # "." adds all changed files, can also add specific files too
git commit -m "[my-commit-message]" # message should be clear and meaningful
git push origin main怎么做
```

CHATGPT

对，这一段就很明确了。你这个 MP0 不需要自己配置普通的 GitHub Pages，也不需要 `npm run build` 后手动上传 `dist`。课程已经给你准备好了 GitHub

Actions: `.github/workflows/static.yml`。

你现在只需要做两件事：

1. GitHub 设置里把 Pages 的 Source 改成 **GitHub Actions**
2. 把你刚才的修改 `git push` 上去

第一步：先把代码推到 GitHub

你刚才 `git status` 显示有：

```
modified:   src/css/main.css
modified:   src/index.html
modified:   src/js/index.js

Untracked files:
src/assets/profile.jpg
```

所以现在在 VS Code Terminal 输入：

```
git add .
```

然后：

```
git status
```

确认这 4 个文件都出现在：

```
Changes to be committed
```

然后：

```
git commit -m "Complete MP0"
```

最后：

```
git push origin main
```

如果 `push` 成功，你的代码就已经上传到 GitHub。

第二步：设置 GitHub Pages

打开你的仓库：

🔗 `https://github.com/Gary22222222/cs409-mp0`

然后：

① 点击 **Settings**

在仓库顶部找到：

```
Code    Issues    Pull requests    Actions    Projects    Security    Insights
Settings
```

点击：

Settings

② 左边找到 **Pages**

进入：

```
Settings
↓
Pages
```

③ 找到 Build and deployment

你会看到：

```
Build and deployment
```

```
Source
```

现在可能是：

```
Deploy from a branch
```

把它改成：

```
GitHub Actions
```

也就是：

```
Source → GitHub Actions
```

保存即可。

第三步：检查 GitHub Actions

然后点击仓库顶部：

Actions

你应该能看到一个 workflow，名字可能类似：

```
Deploy static content to Pages
```

或者来自：

```
.github/workflows/static.yml
```

它会自动运行。

你可能看到：

 In progress

等它变成：

 Success

就说明部署成功。

第四步：打开你的网站

你的用户名是：

Gary22222222

所以按照课程 README：

```
https://Gary22222222.github.io/mp0
```

或者通常 GitHub Pages 会使用小写用户名：

```
https://gary22222222.github.io/mp0
```

课程给的目标格式就是：

```
https://<your-github-username>.github.io/mp0
```

首次部署可能需要 **10-30 分钟**。



你现在最重要的是先做这几个命令

你现在 Terminal 已经在：

```
D:\CS409\cs409-mp0
```

所以直接：

```
git add .
```

↓

```
git commit -m "Complete MP0"
```

↓

```
git push origin main
```

然后去 **GitHub** → **Settings** → **Pages** → **Source** → **GitHub Actions**。

还有一个好消息

你这个仓库已经有：

```
.github/  
└─ workflows/  
    └─ static.yml
```

所以**不要自己创建新的 workflow**。

课程已经替你写好了部署流水线。你只需要：

```
本地修改  
↓  
git add  
↓  
git commit  
↓  
git push  
↓  
GitHub Actions 自动运行  
↓  
GitHub Pages  
↓  
https://gary22222222.github.io/mp0
```

你现在先执行：

```
git add .
git commit -m "Complete MP0"
git push origin main
```

如果最后出现 `main -> main` 之类的成功信息，就把结果发给我，我再告诉你 **GitHub Pages** 页面具体点哪里。

YOU

完成 MP0将静态内容部署到 **Pages** #2: [Gary2222222](#) 推送的 提交 [084640e](#)

[主要的](https://github.com/Gary2222222/cs409-mp0/tree/refs/heads/main "主要的")

svg

3分钟前运行时长 29秒

显示选项

失败的：

初始提交将静态内容部署到页面 #1: [Gary2222222](#) 推送的 提交 [48a779c](#)

[主要的](https://github.com/Gary2222222/cs409-mp0/tree/refs/heads/main "主要的")

svg

今天 18:23运行时长 32秒

显示选项

这个是之前的吧

CHATGPT

对，**这是你 GitHub Actions 的运行记录**，而且从你贴的信息看：

- #1 初始提交 → **失败**
- #2 完成 MP0 → 你刚刚 push 的这次，**目前看起来也是失败/有问题**（你贴出来的页面里显示“失败的”）

所以现在**还不能认为部署成功**。

你现在点进去 完成 MP0 #2

打开你贴的这个：

□url□完成 MP0 #2 Actions□<https://github.com/Gary22222222/cs409-mp0/actions/runs/33283082262>□

进去以后会看到几个步骤，例如：

```
✓ Set up job
✓ Checkout
✗ Build
```

或者：

```
✓ Build
✗ Deploy
```

我们现在最需要看的是红色 ✗ 的那一步。

你可以这样做

进入 完成 MP0 #2 后：

1. 找到带红色 ✗ 的步骤
2. 点击它
3. 把下面出现的**错误信息**复制给我

尤其是类似：

```
Error: ...
```

或者：

```
npm ERR! ...
```

或者：

```
Module not found...
```

关于你问的“这个是之前的吗？”

不是。

你这里有两个运行：

```
#2 完成 MP0  
提交 084640e  
3分钟前
```

这是你刚才提交并 **push** 的代码触发的最新一次。

而：

```
#1 初始提交  
提交 48a779c  
今天 18:23
```

才是之前第一次 **push** 时触发的。

所以我们现在应该看 **#2**。

把 **#2** 点进去后红色 **✖** 那一步的报错贴给我，我可以直接告诉你改哪个文件。

YOU

显示成功了但是<https://gary22222222.github.io/mp0>看不到

CHATGPT

对，如果 **GitHub Actions #2** 显示绿色 **Success**，但 <https://gary22222222.github.io/mp0> 打不开/看不到页面，说明“构建部署成功”与“访问地址正确”之间还有一步需要检查。

我们现在不要再改代码，先检查 **Pages** 的实际网址。

先做这个

打开你的 **GitHub** 仓库：

UrlCs409-mp0 Settings<https://github.com/Gary22222222/cs409-mp0/settings/pages>

找到：

Settings → Pages

在页面上应该有一个类似：

Your site is live at ...

或者：

Visit site

的连接。

直接点击 **GitHub** 给出的这个链接，不要手动输入网址。

如果 Pages 页面显示的是这个

```
https://gary22222222.github.io/cs409-mp0/
```

那么正确地址其实是：

```
https://gary22222222.github.io/cs409-mp0/
```

注意你刚才输入的是：

```
https://gary22222222.github.io/mp0
```

少了：

```
cs409-
```

因为你的 repository 名字是：

```
cs409-mp0
```

所以 GitHub Pages 的默认项目路径通常应该是：

```
/<repository-name>/
```

也就是：

```
/cs409-mp0/
```

你先试这个：

□url□打开你的 MP0 网站□<https://gary22222222.github.io/cs409-mp0/>□

如果这个地址能打开

那就说明：

- ✓ *GitHub Actions 成功*
- ✓ *GitHub Pages 成功*
- ✗ 只是刚才输入的网址少了 `cs409-`

你的最终提交网址就应该是：

```
https://gary22222222.github.io/cs409-mp0/
```

如果这个地址也打不开

那我们再检查 GitHub Pages 的 **Deployment**。

你现在先打开：

□url□GitHub Pages 设置□<https://github.com/Gary22222222/cs409-mp0/settings/pages>□

然后把 **Pages** 页面上显示的内容截图给我。

尤其是这几个地方：

```
Build and deployment
Source
GitHub Actions
```

Your site is live at ...

我就能直接判断是网址问题、Pages 设置问题，还是 workflow 虽然 Success 但实际发布目录不对。

YOU

可以了

CHATGPT

太好了 🍌

那你现在已经完成了整个 MP0 的核心流程：

```
修改 HTML/CSS
↓
本地 localhost 测试
↓
git add .
↓
git commit
↓
git push origin main
↓
GitHub Actions ✅
↓
GitHub Pages ✅
↓
https://gary22222222.github.io/cs409-mp0/
```

你的最终网页地址就是：

📄Url📄你的 MP0 网站📄<https://gary22222222.github.io/cs409-mp0/>📄

接下来记得把课程要求的 **course staff** 添加为 **Collaborator**，然后按照 README 的 submission instructions 提交这个网页地址即可。

另外，你之后如果再修改代码，只需要：

```
git add .
git commit -m "Update MP0"
git push origin main
```

GitHub Actions 会自动重新部署，不需要再手动设置 Pages。

YOU

作为本课程的一部分，我们鼓励学生尝试使用诸如 OpenAI 的 ChatGPT 等 LLM 服务来生成 MP 的源代码。如果学生使用 LLM 生成 MP 的代码，则必须(1) 提交聊天记录以及源代码，(2) 在评分表中回答与使用 LLM 体验相关的调查问卷。未能完成上述步骤将违反本课程的学术诚信政策。这个记录怎么生成

CHATGPT

对，你这次**确实使用了 ChatGPT 帮你完成 MP0 的代码和排错**，所以按照老师这段要求，建议你在这次对话记录一起提交。

你**不需要自己重新整理成 Word**。最稳妥的是提交 **ChatGPT 的聊天记录导出文件**，这样能证明完整的使用过程。

方法一：直接导出 ChatGPT 数据（最完整）

在 ChatGPT 网页端：

1. 点击左下角你的头像/账户
2. 进入 **Settings**
3. 找到 **Data Controls**
4. 找到 **Export Data**
5. 点击 **Export**
6. 确认导出

OpenAI 会把你的聊天数据发送到你的邮箱，里面会包含你的聊天记录。

不过这种方式导出的通常是**整个 ChatGPT 数据包**，可能不太适合直接交给老师。

方法二：只保存这一次 MP0 对话（我更推荐）

你在这个聊天本身就是完整记录。

可以直接使用 ChatGPT 的 **Share / 分享** 功能生成一个只针对这次 conversation 的链接，然后把这个链接按照课程要求提交。

不过要注意：**如果老师明确要求“submit chatlog”作为文件，而不是链接，就不要只交分享链接。**

方法三：打印成 PDF

如果老师接受 PDF，这个非常直观。

在浏览器打开当前这整个对话：

Ctrl + P

然后：

```
Printer / Destination  
↓  
Save as PDF
```

保存成：

```
CS409_MP0_ChatLog.pdf
```

这样里面会包含你和我的完整对话，包括：

- 你问怎么做 MP0
- HTML/CSS 修改
- CSS 没有加载的问题
- webpack 路径问题
- Git 操作
- GitHub Actions
- GitHub Pages 部署

这其实非常适合作为老师要求的 **chatlog**。

 **我建议你不要自己“编一个聊天记录”**

因为老师要求的是：

submit the chatlog and source code

你的真实聊天记录已经存在，所以**直接保存真实记录最好**。

而且你这次的记录其实挺完整的，能够清楚看到：

你自己操作

↓

遇到问题

↓

问 ChatGPT

↓

ChatGPT 给建议

↓

你实际修改

↓

出现 webpack error

↓

继续排错

↓

GitHub Actions 成功

↓

GitHub Pages 成功

这比单独提交一份“我使用了 ChatGPT”的说明更有说服力。

最后

你还需要在 grading form 里回答老师要求的 **LLM experience survey**。这个部分**不要让 ChatGPT 替你虚构体验**，直接根据你这次真实经历回答即可。

如果你愿意，我也可以根据你这次实际完成 MP0 的过程，帮你**逐题解释这个 LLM survey 应该怎么理解、你应该根据真实情况怎么填写**。

YOU

gpt不能部分导出吗

可以，但要分情况。**ChatGPT** 网页目前没有一个特别方便的“只导出当前这一条对话为 PDF/文件”的 **Export Data** 按钮；`Export Data` 通常是导出整个账户的数据。

如果你只想交 **MP0** 这一次聊天记录，我更推荐下面两种：

方法 1：直接把当前对话打印成 PDF ★

在当前这条聊天页面：

Ctrl + P → Save as PDF

这样只会保存当前打开的这条对话页面，不需要把整个 ChatGPT 数据导出来。

文件名可以叫：

```
CS409_MP0_ChatLog.pdf
```

这对于老师说的：

```
submit the chatlog
```

通常是最直观的文件形式。

方法 2：使用 Share

当前对话右上角/顶部菜单里的 **Share (分享)**，可以生成这个 conversation 的分享链接。

但如果作业明确要求 **chatlog** 文件，我会优先交 PDF，而不是只交链接。

如果你担心聊天太长，PDF 变成白板

你之前提到过聊天记录太长导出后出现白板，这种情况下不要直接用整个页面打印。

可以：

1. 把这次 MP0 对话页面滚动到底，让内容加载完整
2. `Ctrl + P`
3. 如果预览仍然白屏，尝试 Chrome 的 **More settings → Background graphics** 打开
4. 或者使用浏览器的 **Save page as / 网页另存为**，再转 PDF

如果你愿意，我还可以告诉你一个**只保存这一个 ChatGPT 对话、而且不会因为聊天太长变白板的办法**。